作文自动评分系统单元测试设计文档

1. 文档目标

本文档基于《作文自动评分系统设计文档》，规划并设计系统的单元测试方案。主要目标包括：

- 明确单元测试的范围、工具与策略。
- 针对核心业务逻辑（如模型推理策略、文本分析、数据读写）设计详细的测试用例。
- 保障系统重构与迭代时的代码质量与稳定性。

2. 测试策略与环境

2.1 测试框架选型

- **后端测试框架**：`pytest`（易于编写和扩展，支持 fixtures）。
- **模拟工具**：`unittest.mock` / `pytest-mock`（用于隔离外部依赖，如跳过真实的 PyTorch 模型加载，模拟 HTTP 请求）。
- **数据库测试**：使用 SQLite 内存数据库 (`:memory:`) 替代本地 `history.db` 文件，确保测试的隔离性与执行速度。
- **前端测试**：`unittest` 或 `pytest` 测试独立的纯函数（如数据解析函数），使用 `requests-mock` 模拟 API 调用。

2.2 测试分层原则

- **无外部依赖**：模型推理部分，应 Mock 掉底层 `transformers` 模型和网络的前向传播，仅测试分数换算、归一化与 `unknown` 融合逻辑。
- **API 测试**：使用 Flask Test Client 进行接口层测试，Mock 掉 `EssayScoringService` 和 `HistoryStore`，专注于请求路由、参数校验和 JSON 响应封装。
- **数据库测试**：直接调用 `HistoryStore` 方法，验证 SQL 执行结果，不经过 Flask 层。

---

3. 测试模块划分与用例设计

3.1 模型推理模块 (`backend/model_service.py`)

本模块包含系统的核心算分策略，特别是 `unknown` 分数的计算逻辑，必须重点覆盖。

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-MDL-01`	**单题评分反标准化**	Mock `_predict_scaled` 返回固定缩放分	text, 存在的 prompt_id	返回合法的 `EssayScoreResult`，真实分数为正确的反标准化值

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-MDL-02`	**无效 prompt_id 拦截**	无	text, 不存在的 prompt_id	抛出 ValueError 或特定异常
`TEST-MDL-03`	**归一化函数校验**	无	各种边界分数 (低于下限、高于上限、正常)	`_normalize_score` 返回 0-1 之间的浮点数，截断越界值
`TEST-MDL-04`	**unknown 评分融合策略**	Mock 多个题目的反标准化得分	text	分别归一化各集合分数后计算均值，最终换算为 `0-100` 分范围，误差 `< 0.01`

3.2 文本分析与 API 模块 (`backend/app.py`)

测试文本特征提取逻辑以及 Flask 路由和参数校验。

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-API-01`	**text_analysis 常规文本**	无	标准中/英文段落（包含多句、多段）	字符数、词数、句子数、段落数正确；返回高频词列表字典
`TEST-API-02`	**text_analysis 边界处理**	无	空字符串 `""` 或纯空白符	各项计数均为 0，不抛出异常
`TEST-API-03`	**单篇评分接口校验**	Mock `EssayScoringService.score`，Flask Client	POST `/api/score/text`，缺省必填参数	HTTP 400，返回明确的参数错误提示
`TEST-API-04`	**单篇评分成功流**	Mock Model 和 Storage，Flask Client	POST `/api/score/text`，合法 JSON	HTTP 200，返回 `submission_id`、`score` 及文本分析结果
`TEST-API-05`	**批量评分文件上传**	Mock 解析与存表，Flask Client	POST `/api/score/file`，附带有效的 CSV 文件	HTTP 200，返回包含每篇作文评分结果的列表

3.3 存储服务模块 (`backend/storage.py`)

验证 SQLite 数据持久化与查询能力。

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-DB-01`	**数据库初始化建表**	内存数据库 `:memory:`	初始化 `HistoryStore`	不抛出异常，查询 `sqlite_master` 包含 `submission` 与 `records` 表

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-DB-02`	**保存单篇与批量记录**	内存数据库已初始化	`save_submission(...)`，包含主表信息及多条明细	返回自增 `submission_id`，数据库中两条相关表的记录数增加对应数量
`TEST-DB-03`	**历史记录分页查询**	数据库插入 15 条测试数据	`get_submissions(page=2, size=10)`	返回 5 条数据，且记录按时间倒序排列
`TEST-DB-04`	**级联删除操作**	已有主表及关联明细记录	`delete_submission(submission_id)`	函数返回 True，再次查询该 ID 主表和明细表均返回为空

3.4 前端工具模块 (`frontend/streamlit_app.py`)

提取前端脱离 Streamlit UI 生命周期的纯函数进行验证，特别是 CSV 解析的容错性。

测试用例 ID	测试目标	前置条件 / 模拟	输入数据	预期输出 / 断言
`TEST-FE-01`	**CSV 解析：正常单列**	构造包含 3 行单列文本的 BytesIO	CSV Bytes	返回包含 3 个字符串的 list
`TEST-FE-02`	**CSV 解析：多列及空单元格**	构造带空白单元格的矩阵 CSV	含有非空及空白 (`""` 或 `NaN`) 的 CSV Bytes	仅提取所有非空单元格的值作为作文，跳过空行空列
`TEST-FE-03`	**CSV 解析：分隔符回退机制**	强行构造逗号分隔文件但破坏自动检测规则	复杂的 CSV Bytes	`sep=None` 失败后回退 `sep=","` 并成功解析
`TEST-FE-04`	**JSON 展平与 CSV 导出**	无	`call_history_detail` 返回的复杂结构字典	`build_export_csv` 成功生成扁平化 CSV 格式文本，高频词转为 JSON 字符串防止格式错乱

---

4. 测试覆盖率目标

- **行覆盖率 (Line Coverage)**：核心服务 `model_service.py` 与 `storage.py` 目标为 **90%** 以上。
- **分支覆盖率 (Branch Coverage)**：针对 `unknown` 评分策略、CSV 回退解析策略，需覆盖所有条件分支（**100%**）。

5. 持续集成与执行建议

1. **环境准备**：创建一个 `tests/` 目录，并按模块建立 `test_model.py`，`test_api.py`，`test_storage.py`。
2. **执行命令**：推荐在项目的 `Makefile` 或执行脚本中添加：

   Bash

   ```
   pytest tests/ -v --cov=backend --cov=frontend --cov-report=html
   ```

3. **大文件绕过**：在测试运行前，利用 Mock 阻断真实模型文件（`best_fold_model.pt`）的加载，保证每次测试能在秒级内执行完毕。